

INSTITUTO FEDERAL DO RIO GRANDE DO NORTE

Campus Natal Central — TADS

Programação e Administração de Banco de Dados (PABD)

Autenticação e Autorização em Django REST Framework

Usuários customizados, perfis e tokens — com e sem JWT

Estudo de caso: API da loja Amazon

Prof. Plácido A. Souza Neto

TADS — 4º Semestre

Sumário

1. Objetivos da Aula

Ao final desta aula, você será capaz de:

- Compreender o que é autenticação e o que é autorização, e por que são conceitos distintos.
- Conhecer as principais formas de autenticação oferecidas pelo Django REST Framework (DRF).
- Criar um modelo de usuário customizado herdando de `AbstractUser`.
- Modelar perfis de usuário de duas maneiras: por campo de escolha (`Choice`) e por classes separadas (`OneToOne`).
- Implementar registro (signup), login e endpoints protegidos usando `TokenAuthentication` (sem JWT).
- Implementar os mesmos fluxos usando JWT com a biblioteca `SimpleJWT`.
- Restringir o acesso a endpoints com base no perfil do usuário (permissões customizadas).

Pré-requisitos

Esta aula dá continuidade ao projeto da API da loja Amazon que construímos ao longo do semestre, com os modelos Cliente, Vendedor e Produto. Aqui vamos repensar o ator “usuário” do sistema e protegê-lo adequadamente.

Pressupõe-se familiaridade com models, serializers, ViewSets e roteamento (routers) do DRF.

2. Autenticação x Autorização

Esses dois termos são frequentemente confundidos, mas representam etapas diferentes do controle de acesso:

Conceito	Pergunta que responde	No DRF
Autenticação	Quem é você?	<code>authentication_classes</code> — identifica o usuário a partir de credenciais (token, sessão, etc.).
Autorização	O que você pode fazer?	<code>permission_classes</code> — decide se o usuário identificado tem permissão para a ação.

Analogia da loja: a autenticação é o porteiro conferindo seu crachá na entrada (ele descobre que você é o vendedor João). A autorização é a regra que diz que João pode cadastrar produtos, mas não pode acessar o painel financeiro da empresa.

3. Formas de Autenticação no DRF

O Django REST Framework oferece várias formas de autenticação. As mais comuns são:

Classe	Como funciona	Quando usar
<code>BasicAuthentication</code>	Envia usuário e senha (codificados em Base64) a cada requisição.	Apenas testes locais. Insegura sem HTTPS.

Classe	Como funciona	Quando usar
SessionAuthentication	Usa o sistema de sessões e cookies do Django.	Quando o cliente é o próprio navegador (mesma origem).
TokenAuthentication	Gera um token opaco no login, armazenado no banco. Enviado no cabeçalho a cada requisição.	APIs simples, apps móveis. Token fica no servidor.
JWTAuthentication	Gera um token assinado (JSON Web Token) que carrega os dados do usuário. Não é armazenado no servidor.	APIs distribuídas/escaláveis, microsserviços, expiração curta + refresh.

Token opaco x JWT — a diferença essencial

TokenAuthentication: o token é só uma chave aleatória. O servidor precisa consultar o banco (tabela `authtoken_token`) para saber a quem ele pertence. Para revogar, basta apagar a linha.

JWT: o token é autocontido — carrega o id do usuário e a validade dentro dele, assinado com uma chave secreta. O servidor valida a assinatura sem ir ao banco. Em troca, revogar um JWT antes da expiração é mais trabalhoso (exige blacklist).

Nesta aula trabalharemos os dois mundos: primeiro TokenAuthentication (mais simples de entender) e depois JWT. E, em cada um, veremos duas formas de modelar o perfil do usuário.

Os 4 cenários desta aula

Cenário A — Perfil por Choice, sem JWT (TokenAuthentication).

Cenário B — Perfil por classes separadas (OneToOne), sem JWT.

Cenário C — Perfil por Choice, com JWT.

Cenário D — Perfil por classes separadas, com JWT.

Todos sobre a mesma loja Amazon, com os atores Cliente e Vendedor.

4. Preparação do Projeto

Partimos do projeto amazon já existente. Vamos garantir as dependências e a configuração base que todos os cenários compartilham.

4.1 Instalando as dependências

```
# Ambiente já criado no semestre; ative-o
source env/bin/activate          # Linux / macOS
env\Scripts\activate            # Windows

# Dependências comuns
pip install django djangorestframework

# Necessária só nos cenários COM JWT (seções 7 e 8)
pip install djangorestframework-simplejwt
```

Terminal — Dependências

4.2 O problema do usuário customizado

O Django já traz um modelo User pronto (username, password, email, first_name, last_name). Para a loja, porém, precisamos distinguir Cliente de Vendedor e guardar campos extras como CPF e telefone.

Defina o usuário customizado ANTES da primeira migração

Trocar o modelo de usuário depois que o banco já foi migrado é doloroso (envolve apagar migrações e o banco). Por isso, sempre que um projeto possa precisar de campos extras no usuário, comece já com um AbstractUser próprio.

A regra de ouro: configure AUTH_USER_MODEL no primeiro dia do projeto.

4.3 Configuração base do settings.py

Esta configuração é comum aos cenários A e B (sem JWT). Os ajustes específicos de JWT virão depois.

```
# amazon/settings.py

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'rest_framework',
    'rest_framework.authtoken',  # tabela de tokens (TokenAuthentication)
    'backend',                  # app da loja Amazon
]

# Diz ao Django para usar nosso usuário customizado
AUTH_USER_MODEL = 'backend.Usuario'

REST_FRAMEWORK = {
```

```
'DEFAULT_AUTHENTICATION_CLASSES': [  
    'rest_framework.authentication.TokenAuthentication',  
    'rest_framework.authentication.SessionAuthentication',  
],  
'DEFAULT_PERMISSION_CLASSES': [  
    'rest_framework.permissions.IsAuthenticated',  
],  
}
```

amazon/settings.py — configuração base (sem JWT)

Por que registrar `rest_framework.authtoken`?

Essa app cria a tabela `authtoken_token`, onde cada token gerado fica associado a um usuário. Sem ela, `Token.objects.create(...)` falha. Lembre de rodar as migrações após adicioná-la.

5. Cenário A — Perfil por Choice (sem JWT)

Neste primeiro cenário, o usuário herda de `AbstractUser` e o seu papel na loja é definido por um único campo de escolha (tipo): `CLIENTE` ou `VENDEDOR`. É a abordagem mais simples e direta.

Quando usar a abordagem por Choice

Use quando os diferentes perfis NÃO precisam de campos próprios muito distintos. Se Cliente e Vendedor têm praticamente os mesmos atributos e mudam só no “papel”, um campo tipo resolve com elegância.

5.1 O modelo Usuario com Choice

Criamos uma classe interna `TextChoices` para listar os papéis possíveis e um campo tipo que os referencia. Métodos auxiliares (`is_cliente`, `is_vendedor`) deixam o código de permissões mais legível.

```
# backend/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models

class Usuario(AbstractUser):
    class Tipo(models.TextChoices):
        CLIENTE = 'CLIENTE', 'Cliente'
        VENDEDOR = 'VENDEDOR', 'Vendedor'

    tipo = models.CharField(
        max_length=10,
        choices=Tipo.choices,
        default=Tipo.CLIENTE,
    )
    cpf = models.CharField(max_length=11, unique=True)
    telefone = models.CharField(max_length=15, blank=True)

    def is_cliente(self):
        return self.tipo == self.Tipo.CLIENTE

    def is_vendedor(self):
        return self.tipo == self.Tipo.VENDEDOR

    def __str__(self):
        return f'{self.username} ({self.get_tipo_display()})'
```

backend/models.py — Usuario com campo de escolha

get_tipo_display()

Para todo campo com choices, o Django cria automaticamente o método `get_<campo>_display()`, que devolve o rótulo legível (“Vendedor”) em vez do valor armazenado (“VENDEDOR”). Muito útil em telas e logs.

5.2 Serializers

Precisamos de dois serializers: um para o registro (recebe a senha, `write_only`) e outro, opcional, para exibição. O campo `password` nunca deve ser devolvido na resposta.

```
# backend/serializers.py
from rest_framework import serializers
from .models import Usuario

class UsuarioSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True)

    class Meta:
        model = Usuario
        fields = ['id', 'username', 'email', 'password',
                  'tipo', 'cpf', 'telefone']

    def create(self, validated_data):
        # create_user faz o hash da senha corretamente
        senha = validated_data.pop('password')
        usuario = Usuario(**validated_data)
        usuario.set_password(senha)
        usuario.save()
        return usuario
```

backend/serializers.py — UsuarioSerializer (Cenário A)

Nunca salve a senha em texto puro!

Repare no uso de `set_password(senha)`: ele aplica o algoritmo de hash do Django (PBKDF2 por padrão). Se você fizesse `Usuario.objects.create(password=...)`, a senha ficaria salva em texto puro e o login com `check_password()` jamais funcionaria. Este é o erro nº 1 dos iniciantes.

5.3 Views — signup, login e endpoint protegido

No fluxo sem JWT, o login gera (ou recupera) um token da tabela `authtoken_token` e o devolve ao cliente. Os endpoints protegidos exigem esse token no cabeçalho.

```
# backend/views.py
from rest_framework import status
from rest_framework.decorators import (api_view, authentication_classes,
                                       permission_classes)

from rest_framework.response import Response
from rest_framework.authtoken.models import Token
from rest_framework.authentication import TokenAuthentication
from rest_framework.permissions import IsAuthenticated, AllowAny
from django.shortcuts import get_object_or_404
from .models import Usuario
from .serializers import UsuarioSerializer

@api_view(['POST'])
@permission_classes([AllowAny])          # registro é público
def signup(request):
    serializer = UsuarioSerializer(data=request.data)
    if serializer.is_valid():
        usuario = serializer.save()      # chama o create() do serializer
        token = Token.objects.create(user=usuario)
        return Response({'token': token.key, 'usuario': serializer.data},
                        status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

```

@api_view(['POST'])
@permission_classes([AllowAny])          # login é público
def login(request):
    usuario = get_object_or_404(Usuario, username=request.data.get('username'))
    if not usuario.check_password(request.data.get('password')):
        return Response({'detail': 'Credenciais inválidas.'},
                        status=status.HTTP_400_BAD_REQUEST)
    token, _ = Token.objects.get_or_create(user=usuario)
    return Response({'token': token.key,
                    'usuario': UsuarioSerializer(usuario).data})

@api_view(['GET'])
@authentication_classes([TokenAuthentication])
@permission_classes([IsAuthenticated])  # exige token válido
def perfil(request):
    return Response({
        'username': request.user.username,
        'tipo': request.user.tipo,
        'mensagem': f'Olá, {request.user.username}! Você é
{request.user.get_tipo_display()}'
    })

```

backend/views.py — fluxo de autenticação (Cenário A)

5.4 Autorização por perfil — permissão customizada

Agora a autorização: só um **VENDEDOR** pode cadastrar produtos na loja. Criamos uma classe de permissão reutilizável.

```

# backend/permissions.py
from rest_framework.permissions import BasePermission

class IsVendedor(BasePermission):
    message = 'Apenas vendedores podem realizar esta ação.'

    def has_permission(self, request, view):
        return bool(request.user and request.user.is_authenticated
                    and request.user.is_vendedor())

```

backend/permissions.py — permissão por perfil

Aplicando a permissão a um ViewSet de produtos:

```

# backend/views.py (continuação)
from rest_framework import viewsets
from .permissions import IsVendedor
from .models import Produto
from .serializers import ProdutoSerializer

class ProdutoViewSet(viewsets.ModelViewSet):
    queryset = Produto.objects.all()
    serializer_class = ProdutoSerializer

    def get_permissions(self):
        # Qualquer autenticado pode listar/ver; só vendedor escreve
        if self.action in ['list', 'retrieve']:
            return [IsAuthenticated()]
        return [IsAuthenticated(), IsVendedor()]

```

backend/views.py — ViewSet protegido por perfil

5.5 Rotas

```
# backend/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views

router = DefaultRouter()
router.register('produtos', views.ProdutoViewSet, basename='produto')

urlpatterns = [
    path('signup/', views.signup),
    path('login/', views.login),
    path('perfil/', views.perfil),
    path('', include(router.urls)),
]
```

backend/urls.py — Cenário A

5.6 Migrando e testando

```
python manage.py makemigrations
python manage.py migrate
```

Terminal — migrações

Arquivo de teste no formato REST Client (.http). Repare que, após o login, copiamos o token para o cabeçalho Authorization no formato Token <chave>:

```
### 1) Registrar um vendedor
POST http://localhost:8000/api/signup/
Content-Type: application/json

{
  "username": "joao_vendedor",
  "email": "joao@loja.com",
  "password": "senha123",
  "tipo": "VENDEDOR",
  "cpf": "12345678901",
  "telefone": "84999990000"
}

### 2) Login (devolve o token)
POST http://localhost:8000/api/login/
Content-Type: application/json

{
  "username": "joao_vendedor",
  "password": "senha123"
}

### 3) Acessar endpoint protegido (cole o token recebido)
GET http://localhost:8000/api/perfil/
Authorization: Token 9a1b2c3d4e5f...

### 4) Cadastrar produto (só vendedor consegue)
POST http://localhost:8000/api/produtos/
Authorization: Token 9a1b2c3d4e5f...
Content-Type: application/json
```

```
{  
  "nome": "Echo Dot 5",  
  "preco": "299.90"  
}
```

api.http — testes do Cenário A

O que esperar

Um cliente (tipo CLIENTE) que tente o passo 4 receberá 403 Forbidden com a mensagem “Apenas vendedores podem realizar esta ação.” — autorização funcionando. Uma requisição sem o cabeçalho Authorization receberá 401 Unauthorized — autenticação funcionando.

6. Cenário B — Perfis como classes separadas (sem JWT)

Aqui o usuário continua herdando de `AbstractUser`, mas o papel deixa de ser um simples campo. Cada perfil vira uma classe própria — `PerfilCliente` e `PerfilVendedor` — ligada ao usuário por um relacionamento `OneToOne`. Assim, cada perfil pode ter seus próprios campos.

Quando usar a abordagem por classes

Use quando `Cliente` e `Vendedor` têm dados realmente diferentes. Na loja Amazon: o `Cliente` tem endereço de entrega e data de nascimento; o `Vendedor` tem nome da loja, CNPJ e avaliação média. Forçar tudo num único modelo geraria muitas colunas nulas.

6.1 Os modelos

Mantemos no `Usuario` apenas o campo `tipo` (para sabermos rapidamente qual perfil buscar) e os dados comuns. Os detalhes vão para os perfis.

```
# backend/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models

class Usuario(AbstractUser):
    class Tipo(models.TextChoices):
        CLIENTE = 'CLIENTE', 'Cliente'
        VENDEDOR = 'VENDEDOR', 'Vendedor'

    tipo = models.CharField(max_length=10, choices=Tipo.choices)
    telefone = models.CharField(max_length=15, blank=True)

    def is_vendedor(self):
        return self.tipo == self.Tipo.VENDEDOR

class PerfilCliente(models.Model):
    usuario = models.OneToOneField(
        Usuario, on_delete=models.CASCADE, related_name='perfil_cliente')
    cpf = models.CharField(max_length=11, unique=True)
    endereco_entrega = models.CharField(max_length=255)
    data_nascimento = models.DateField(null=True, blank=True)

    def __str__(self):
        return f'Cliente: {self.usuario.username}'

class PerfilVendedor(models.Model):
    usuario = models.OneToOneField(
        Usuario, on_delete=models.CASCADE, related_name='perfil_vendedor')
    nome_loja = models.CharField(max_length=120)
    cnpj = models.CharField(max_length=14, unique=True)
    avaliacao_media = models.DecimalField(
        max_digits=3, decimal_places=2, default=0)

    def __str__(self):
        return f'Vendedor: {self.nome_loja}'
```

backend/models.py — Usuario + perfis OneToOne (Cenário B)

related_name e o acesso reverso

Graças ao `related_name`, a partir de um usuário vendedor podemos fazer `usuario.perfil_vendedor.nome_loja`. Se o usuário não tiver aquele perfil, o acesso lança a exceção `RelatedObjectDoesNotExist` — por isso é prudente checar o tipo antes, ou usar `hasattr(usuario, 'perfil_vendedor')`.

6.2 Serializers aninhados

O serializer de registro recebe, além dos dados do usuário, o bloco do perfil correspondente. No `create()`, criamos o usuário e, em seguida, o perfil ligado a ele — tudo dentro de uma transação para garantir atomicidade.

```
# backend/serializers.py
from rest_framework import serializers
from django.db import transaction
from .models import Usuario, PerfilCliente, PerfilVendedor

class PerfilClienteSerializer(serializers.ModelSerializer):
    class Meta:
        model = PerfilCliente
        fields = ['cpf', 'endereco_entrega', 'data_nascimento']

class PerfilVendedorSerializer(serializers.ModelSerializer):
    class Meta:
        model = PerfilVendedor
        fields = ['nome_loja', 'cnpj', 'avaliacao_media']
        read_only_fields = ['avaliacao_media']

class RegistroSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True)
    # Os dois perfis são opcionais; o cliente envia apenas o seu
    perfil_cliente = PerfilClienteSerializer(required=False)
    perfil_vendedor = PerfilVendedorSerializer(required=False)

    class Meta:
        model = Usuario
        fields = ['id', 'username', 'email', 'password', 'tipo',
                  'telefone', 'perfil_cliente', 'perfil_vendedor']

    def validate(self, data):
        tipo = data.get('tipo')
        if tipo == Usuario.Tipo.CLIENTE and 'perfil_cliente' not in data:
            raise serializers.ValidationError(
                'Cliente precisa enviar perfil_cliente.')
        if tipo == Usuario.Tipo.VENDEDOR and 'perfil_vendedor' not in data:
            raise serializers.ValidationError(
                'Vendedor precisa enviar perfil_vendedor.')
        return data

    @transaction.atomic
    def create(self, validated_data):
        dados_cliente = validated_data.pop('perfil_cliente', None)
        dados_vendedor = validated_data.pop('perfil_vendedor', None)
        senha = validated_data.pop('password')
```

```

usuario = Usuario(**validated_data)
usuario.set_password(senha)
usuario.save()

if dados_cliente:
    PerfilCliente.objects.create(usuario=usuario, **dados_cliente)
if dados_vendedor:
    PerfilVendedor.objects.create(usuario=usuario, **dados_vendedor)
return usuario

```

backend/serializers.py — registro com perfil aninhado (Cenário B)

@transaction.atomic

Se a criação do perfil falhar depois que o usuário já foi salvo, ficaríamos com um usuário órfão (sem perfil). O decorador `@transaction.atomic` garante que ou tudo é gravado, ou nada é — o banco volta ao estado anterior em caso de erro.

6.3 Views

O signup e o login são muito parecidos com o Cenário A — a diferença está no serializer usado. O endpoint de perfil agora monta a resposta a partir da classe de perfil correta.

```

# backend/views.py
from rest_framework import status
from rest_framework.decorators import (api_view, authentication_classes,
                                       permission_classes)

from rest_framework.response import Response
from rest_framework.authtoken.models import Token
from rest_framework.authentication import TokenAuthentication
from rest_framework.permissions import IsAuthenticated, AllowAny
from django.shortcuts import get_object_or_404
from .models import Usuario
from .serializers import (RegistroSerializer, PerfilClienteSerializer,
                          PerfilVendedorSerializer)

@api_view(['POST'])
@permission_classes([AllowAny])
def signup(request):
    serializer = RegistroSerializer(data=request.data)
    if serializer.is_valid():
        usuario = serializer.save()
        token = Token.objects.create(user=usuario)
        return Response({'token': token.key, 'usuario': serializer.data},
                        status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['POST'])
@permission_classes([AllowAny])
def login(request):
    usuario = get_object_or_404(Usuario, username=request.data.get('username'))
    if not usuario.check_password(request.data.get('password')):
        return Response({'detail': 'Credenciais inválidas.'},
                        status=status.HTTP_400_BAD_REQUEST)
    token, _ = Token.objects.get_or_create(user=usuario)
    return Response({'token': token.key, 'tipo': usuario.tipo})

@api_view(['GET'])

```

```
@authentication_classes([TokenAuthentication])
@permission_classes([IsAuthenticated])
def perfil(request):
    usuario = request.user
    if usuario.is_vendedor() and hasattr(usuario, 'perfil_vendedor'):
        dados = PerfilVendedorSerializer(usuario.perfil_vendedor).data
    elif hasattr(usuario, 'perfil_cliente'):
        dados = PerfilClienteSerializer(usuario.perfil_cliente).data
    else:
        dados = {}
    return Response({'username': usuario.username,
                    'tipo': usuario.tipo, 'perfil': dados})
```

backend/views.py — Cenário B

6.4 Teste de registro com perfil aninhado

```
### Registrar um vendedor COM perfil de vendedor
POST http://localhost:8000/api/signup/
Content-Type: application/json

{
  "username": "loja_tech",
  "email": "contato@lojatech.com",
  "password": "senha123",
  "tipo": "VENDEDOR",
  "telefone": "84999990000",
  "perfil_vendedor": {
    "nome_loja": "Loja Tech RN",
    "cnpj": "12345678000199"
  }
}

### Registrar um cliente COM perfil de cliente
POST http://localhost:8000/api/signup/
Content-Type: application/json

{
  "username": "maria_cliente",
  "email": "maria@email.com",
  "password": "senha123",
  "tipo": "CLIENTE",
  "perfil_cliente": {
    "cpf": "98765432100",
    "endereco_entrega": "Av. Salgado Filho, 1000 - Natal/RN"
  }
}
```

api.http — registro aninhado (Cenário B)

Choice x Classes — qual escolher?

Choice (Cenário A): mais simples, menos tabelas, ótimo quando os perfis quase não diferem em campos.

Classes OneToOne (Cenário B): mais flexível e limpo quando cada perfil tem atributos próprios; evita colunas nulas e mantém cada responsabilidade no seu modelo.

Repare que a camada de autenticação (token, login) é idêntica nos dois — o que muda é a modelagem de dados, não a segurança.

7. Migrando para JWT

Até aqui usamos `TokenAuthentication`: um token opaco guardado no banco. Vamos agora trocar o mecanismo de autenticação por JWT (JSON Web Token), mantendo a mesma loja e os mesmos modelos. A boa notícia: a modelagem de usuário e perfil não muda — só muda como o token é emitido e validado.

7.1 Anatomia de um JWT

Um JWT é uma string com três partes separadas por pontos (`header.payload.signature`):

- **Header:** diz o algoritmo de assinatura (ex.: HS256).
- **Payload:** carrega os “claims” — id do usuário, data de expiração, etc. NÃO é criptografado, apenas codificado em Base64; portanto, nunca coloque segredos aqui.
- **Signature:** assinatura gerada com a `SECRET_KEY` do servidor. Garante que o token não foi adulterado.

Payload é legível por qualquer um

Qualquer pessoa pode decodificar o payload de um JWT (por exemplo, em jwt.io) e ler seu conteúdo. A assinatura impede a falsificação, mas não esconde os dados. Nunca inclua senhas ou dados sensíveis no token.

7.2 Access token e refresh token

O `SimpleJWT` trabalha com dois tokens. Entender a dupla é essencial:

Token	Validade típica	Função
access	Curta (5 a 60 min)	Enviado a cada requisição protegida. Se vaziar, expira logo.
refresh	Longa (1 a 7 dias)	Usado só para obter um novo access quando este expira, sem refazer login.

8. Cenário C — Perfil por Choice (com JWT)

Reaproveitamos exatamente o modelo `Usuario` com campo tipo do Cenário A. Só trocamos a infraestrutura de autenticação.

8.1 Ajustes no `settings.py`

```
# amazon/settings.py
from datetime import timedelta

INSTALLED_APPS = [
    # ... apps do Django ...
    'rest_framework',
    'rest_framework_simplejwt',  # <-- adicionado
    'backend',
]

AUTH_USER_MODEL = 'backend.Usuario'
```

```

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),
    'ROTATE_REFRESH_TOKENS': False,
}

```

amazon/settings.py — configuração JWT

Não precisa mais de authtoken

Com JWT, a app `rest_framework.authtoken` e a tabela de tokens não são necessárias — o token não é guardado no banco. Você pode removê-la se o projeto for usar apenas JWT.

8.2 Rotas de token prontas do SimpleJWT

O SimpleJWT já oferece views prontas para emitir e renovar tokens. Não precisamos escrever a view de login manualmente — basta registrar as rotas.

```

# backend/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from rest_framework_simplejwt.views import (
    TokenObtainPairView, # login: devolve access + refresh
    TokenRefreshView,   # renova o access usando o refresh
)
from . import views

router = DefaultRouter()
router.register('produtos', views.ProdutoViewSet, basename='produto')

urlpatterns = [
    path('signup/', views.signup),
    path('login/', TokenObtainPairView.as_view(), name='login'),
    path('refresh/', TokenRefreshView.as_view(), name='refresh'),
    path('perfil/', views.perfil),
    path('', include(router.urls)),
]

```

backend/urls.py — rotas JWT (Cenário C)

8.3 Customizando o token para incluir o perfil

Por padrão, o login do SimpleJWT só recebe username e password. Muitas vezes queremos que o próprio token (ou a resposta do login) carregue o tipo do usuário, para o frontend já saber qual interface mostrar. Fazemos isso com um serializer customizado.

```

# backend/serializers.py
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer

```

```
class LoginSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        # claims extras embutidos no payload do token
        token['tipo'] = user.tipo
        token['username'] = user.username
        return token

    def validate(self, attrs):
        data = super().validate(attrs)
        # campos extras na RESPOSTA do login (fora do token)
        data['tipo'] = self.user.tipo
        return data
```

backend/serializers.py — login JWT com claim de perfil

E a view de login passa a usar esse serializer:

```
# backend/views.py
from rest_framework_simplejwt.views import TokenObtainPairView
from .serializers import LoginSerializer

class LoginView(TokenObtainPairView):
    serializer_class = LoginSerializer

# em urls.py, troque por:
# path('login/', LoginView.as_view(), name='login'),
```

backend/views.py — view de login customizada

8.4 Signup, perfil e permissão

O signup continua sendo nosso (o SimpleJWT não cria usuários). Reaproveitamos o UsuarioSerializer do Cenário A, mas agora, após criar o usuário, devolvemos um par de tokens JWT em vez de um token opaco.

```
# backend/views.py
from rest_framework import status, viewsets
from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated, AllowAny
from rest_framework_simplejwt.tokens import RefreshToken
from .serializers import UsuarioSerializer
from .permissions import IsVendedor
from .models import Produto
from .serializers import ProdutoSerializer

@api_view(['POST'])
@permission_classes([AllowAny])
def signup(request):
    serializer = UsuarioSerializer(data=request.data)
    if serializer.is_valid():
        usuario = serializer.save()
        refresh = RefreshToken.for_user(usuario) # gera o par
        return Response({
            'refresh': str(refresh),
            'access': str(refresh.access_token),
            'usuario': serializer.data,
```

```

    }, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET'])
@permission_classes([IsAuthenticated]) # JWT já é o default agora
def perfil(request):
    return Response({
        'username': request.user.username,
        'tipo': request.user.tipo,
        'mensagem': f'Autenticado via JWT como {request.user.get_tipo_display()}.'
    })

class ProdutoViewSet(viewsets.ModelViewSet):
    queryset = Produto.objects.all()
    serializer_class = ProdutoSerializer
    def get_permissions(self):
        if self.action in ['list', 'retrieve']:
            return [IsAuthenticated()]
        return [IsAuthenticated(), IsVendedor()]

```

backend/views.py — signup + perfil com JWT (Cenário C)

A permissão IsVendedor não muda

A classe IsVendedor criada no Cenário A funciona igual aqui. A autorização (o que o usuário pode fazer) é independente do mecanismo de autenticação (como ele provou quem é). Trocar Token por JWT não afeta as regras de permissão.

8.5 Testando o fluxo JWT

Agora o cabeçalho usa o esquema Bearer (e não Token). Note também o endpoint de refresh.

```

### 1) Login – devolve access e refresh
POST http://localhost:8000/api/login/
Content-Type: application/json

{
  "username": "joao_vendedor",
  "password": "senha123"
}

### 2) Endpoint protegido – esquema Bearer
GET http://localhost:8000/api/perfil/
Authorization: Bearer eyJhbGciOiJIUzI1NiI...

### 3) Renovar o access quando expirar
POST http://localhost:8000/api/refresh/
Content-Type: application/json

{
  "refresh": "eyJhbGciOiJIUzI1NiI..."
}

```

api.http — fluxo JWT (Cenário C)

Token vs Bearer — erro comum

Com TokenAuthentication o cabeçalho é Authorization: Token <chave>. Com JWT é Authorization: Bearer <token>. Trocar um pelo outro resulta em 401. Atenção redobrada ao migrar os testes.

9. Cenário D — Perfis como classes separadas (com JWT)

Combinamos a modelagem rica do Cenário B (PerfilCliente e PerfilVendedor via OneToOne) com a autenticação JWT do Cenário C. Reaproveitamos os modelos e o RegistroSerializer do Cenário B sem alteração — apenas o signup devolve JWT e o login passa a embutir dados do perfil.

9.1 Login com dados do perfil no token

Como cada perfil é uma classe própria, podemos enriquecer a resposta do login com informações específicas — por exemplo, o nome da loja do vendedor.

```
# backend/serializers.py
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer

class LoginSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        token['tipo'] = user.tipo
        return token

    def validate(self, attrs):
        data = super().validate(attrs)
        user = self.user
        data['tipo'] = user.tipo
        if user.is_vendedor() and hasattr(user, 'perfil_vendedor'):
            data['nome_loja'] = user.perfil_vendedor.nome_loja
        elif hasattr(user, 'perfil_cliente'):
            data['cpf'] = user.perfil_cliente.cpf
        return data
```

backend/serializers.py — login JWT com perfil (Cenário D)

9.2 Signup devolvendo JWT

```
# backend/views.py
from rest_framework import status
from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework.permissions import AllowAny, IsAuthenticated
from rest_framework_simplejwt.tokens import RefreshToken
from rest_framework_simplejwt.views import TokenObtainPairView
from .serializers import RegistroSerializer, LoginSerializer,
    PerfilClienteSerializer, PerfilVendedorSerializer

class LoginView(TokenObtainPairView):
    serializer_class = LoginSerializer

@api_view(['POST'])
@permission_classes([AllowAny])
def signup(request):
    serializer = RegistroSerializer(data=request.data)
    if serializer.is_valid():
        usuario = serializer.save()
        refresh = RefreshToken.for_user(usuario)
        return Response({
            'refresh': str(refresh),
```

```

        'access': str(refresh.access_token),
        'usuario': serializer.data,
    }, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def perfil(request):
    usuario = request.user
    if usuario.is_vendedor() and hasattr(usuario, 'perfil_vendedor'):
        dados = PerfilVendedorSerializer(usuario.perfil_vendedor).data
    elif hasattr(usuario, 'perfil_cliente'):
        dados = PerfilClienteSerializer(usuario.perfil_cliente).data
    else:
        dados = {}
    return Response({'username': usuario.username,
                    'tipo': usuario.tipo, 'perfil': dados})

```

backend/views.py — Cenário D

Reaproveitamento de código

Compare os Cenários B e D: os modelos, o RegistroSerializer (com create() transacional) e a view de perfil são idênticos. A única mudança real é o emissor do token (RefreshToken.for_user em vez de Token.objects.create) e a classe de autenticação. Essa separação de responsabilidades é o que torna o código sustentável.

9.3 Teste

```

### Registro de vendedor com perfil – devolve JWT
POST http://localhost:8000/api/signup/
Content-Type: application/json

{
  "username": "loja_tech",
  "email": "contato@lojatech.com",
  "password": "senha123",
  "tipo": "VENDEDOR",
  "perfil_vendedor": {
    "nome_loja": "Loja Tech RN",
    "cnpj": "12345678000199"
  }
}

### Endpoint protegido
GET http://localhost:8000/api/perfil/
Authorization: Bearer eyJhbGciOiJIUzI1NiI...

```

api.http — Cenário D

10. Comparação dos Quatro Cenários

A tabela abaixo resume as diferenças entre os quatro cenários trabalhados. Note como autenticação e modelagem de perfil são eixos independentes — você pode combinar qualquer modelagem com qualquer mecanismo.

	Sem JWT (Token)	Com JWT (SimpleJWT)
Perfil por Choice	Cenário A — campo tipo + Token no banco.	Cenário C — campo tipo + access/refresh.
Perfil por Classes	Cenário B — OneToOne + Token no banco.	Cenário D — OneToOne + access/refresh.

10.1 Eixo de autenticação

Aspecto	TokenAuthentication	JWT
Armazenamento	Token guardado no banco (authtoken_token).	Nada guardado; token autocontido.
Cabeçalho	Authorization: Token <chave>	Authorization: Bearer <token>
Expiração	Não expira por padrão.	Access expira; refresh renova.
Revogação	Simples: apagar a linha do token.	Difícil: exige blacklist.
Escalabilidade	Consulta o banco a cada requisição.	Valida só a assinatura; mais escalável.
Login pronto	Escrito manualmente.	TokenObtainPairView pronto.

10.2 Eixo de modelagem de perfil

Aspecto	Choice (campo tipo)	Classes OneToOne
Complexidade	Baixa — um único modelo.	Maior — um modelo por perfil.
Campos por perfil	Compartilhados; gera colunas nulas.	Cada perfil tem os seus.
Consulta do papel	usuario.tipo	usuario.perfil_vendedor / perfil_cliente
Melhor para	Perfis semelhantes.	Perfis com dados distintos.

11. Boas Práticas de Segurança

- **Sempre HTTPS em produção.** Qualquer token (opaco ou JWT) trafegando em HTTP pode ser interceptado.
- **set_password, nunca senha em texto puro.** Deixe o Django cuidar do hash.
- **Access tokens de vida curta.** Reduz a janela de exposição em caso de vazamento.
- **Nunca coloque segredos no payload do JWT.** Ele é apenas codificado, não criptografado.

- **Valide a senha no registro.** Use `validate_password` do Django para exigir senhas fortes.
- **Princípio do menor privilégio.** Conceda a cada perfil apenas as permissões estritamente necessárias.